

Architecture, Design & Delivery Plan

Privacy-First Asymmetric Legal Risk Quantifier

Version 1.0 | Companion to PRD v1.0 and TRD v1.0

Contains: architecture.md · design.md · phase-wise plan · test plan · learning roadmap · suggestions

Prepared for: Kanak (kankuu) | Date: August 2026

Contents

1. architecture.md — System Architecture
2. design.md — Key Design Decisions
3. Phase-Wise Build Plan
4. Things to Learn (Mapped to Phases)
5. Test Plan
6. Suggestions

1. architecture.md

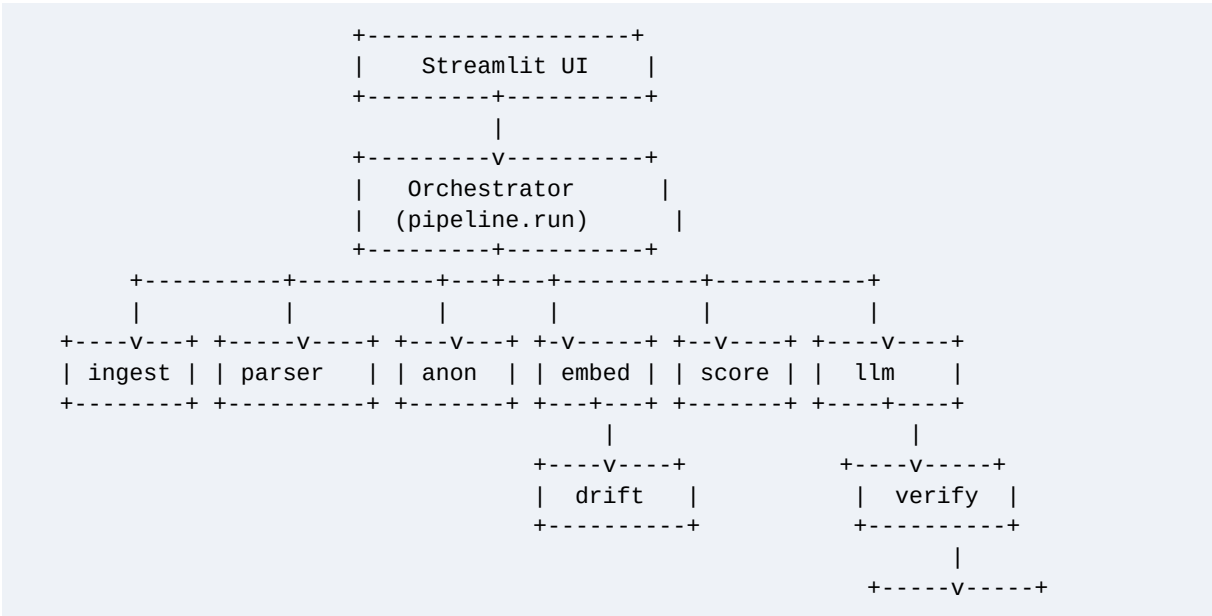
1.1 System Context

A single-user, fully local application. The user interacts only with the Streamlit UI; everything else runs as in-process Python modules or a localhost Ollama server. There is no external backend and no cloud dependency at runtime (only at one-time setup, to pull model weights).

1.2 Module Boundaries

Module	Owns	Depends on
ingest/	File upload handling, format detection (PDF/DOCX)	pdfplumber, PyMuPDF, python-docx
parser/	Layout analysis & rarr; structured clause tree	ingest/
anonymize/	NER + entity masking + entity_map	parser/, spaCy
embed/	Clause vectorization + caching	anonymize/, sentence-transformers
drift/	Cross-version clause matching + similarity classification	embed/
score/	Role-weight matrix + risk scoring	parser/ (category), config/
llm/	Prompt construction, Ollama client, strict JSON parsing	anonymize/ (masked text only)
verify/	Anchor verification against source tree	parser/, llm/
eval/	CUAD loader, metric computation, adversarial hallucination tests	all of the above
ui/	Streamlit app, session state, rendering	everything above, via a single orchestrator function
config/	YAML/TOML for thresholds, model name, weight matrix	none

1.3 Architecture Diagram (textual)



1.4 Key Architectural Principles

- **Anonymize-before-generate:** the LLM module physically cannot receive raw text; the anonymizer sits between the parser and every downstream consumer.
- **Deterministic gate on generative output:** nothing generated by the LLM reaches the UI without passing through verify/ first — this is a hard pipeline stage, not an optional check.
- **Config-driven thresholds:** similarity cutoffs and role weights live in config, not code, so they can be calibrated (Phase 5) without redeploying.
- **Local-only boundary:** the only module allowed to open a network socket is the Ollama client, and only to localhost.

2. design.md

2.1 Why a Rule-Based Layout Parser Instead of Fixed-Size Chunking

Fixed-size or token-count chunking (as used by most RAG pipelines) destroys legal hierarchy: a sub-clause can be split from its parent obligation, changing its meaning. Using font-size, boldness, and numbering-pattern heuristics to rebuild a tree preserves the parent-child relationship that determines which obligations a given sentence actually falls under. Trade-off: this heuristic parser will need per-document-style tuning; the design.md explicitly accepts a documented fallback (paragraph chunking with a lower-confidence flag) rather than trying to handle every possible document style perfectly in v1.

2.2 Why Deterministic Anchor Verification, Not a Second LLM Call

It is tempting to "double check" one LLM with another LLM call, but that only reduces the probability of hallucination, it does not eliminate it, and two models can share correlated blind spots. A regex/string match against the actual source tree is either true or false; it cannot hallucinate. This is the core reliability guarantee of the whole product and is worth the extra engineering effort of maintaining exact character-span tracking through the pipeline.

2.3 Why Role Weighting Is a Matrix, Not a Prompt Instruction

Simply telling the LLM "the user is a Client, weigh accordingly" makes the scoring unexplainable and inconsistent across runs. A hand-authored (then data-refined) matrix means every score can be traced to a specific, inspectable rule — important both for user trust and because it is the kind of design decision that reads well in a technical interview.

2.4 Why Clause-Level Embeddings Instead of Whole-Document Embeddings

Whole-document similarity would hide a single dangerous change inside thousands of unchanged words. Clause-level vectors, matched pairwise across versions, let the drift engine isolate exactly which clause moved and by how much — this is what makes "intent diffing" meaningfully different from textual diffing.

2.5 State Management in the UI

Streamlit re-runs the whole script on every interaction, so parsed trees, embeddings, and eval results should be cached in `st.session_state` keyed by a content hash of the uploaded file, to avoid re-running the full pipeline (including the LLM calls) on every UI click.

2.6 Error/Fallback Design

Failure	Fallback Behavior
Parser cannot detect clear heading structure	Fall back to paragraph-boundary chunking; mark tree with confidence = low
LLM returns malformed JSON	One retry with a stricter format reminder; else mark clause 'unclassified', never fabricate
Anchor verification fails	Drop the claim silently from the UI; log it to a local debug file for developer review
Ollama server not running	Show a clear setup instruction in the UI rather than a stack trace

3. Phase-Wise Build Plan

Sized for solo, self-study development at roughly 2-3 focused hours/day. Adjust pace as needed — the sequencing matters more than the exact day counts.

Phase 0: Setup & Scaffolding (2-3 days)

- Repo scaffold with the module layout from architecture.md (ingest/, parser/, anonymize/, embed/, drift/, score/, llm/, verify/, eval/, ui/, config/).
- Install & smoke-test Ollama with a quantized Llama-3-8B-Instruct model.
- Set up requirements.txt, a config.yaml stub, and pytest scaffolding.
- Collect 3-5 sample contracts/policies (and, if possible, a matched before/after pair) for manual testing.

Phase 1: Structural Layout Parser (4-6 days)

- Implement PDF text + font-metadata extraction (pdfplumber/PyMuPDF).
- Build the heading/level classification heuristic and construct the nested clause-tree JSON.
- Implement the paragraph-chunking fallback path for low-confidence documents.
- Unit tests on 3+ real sample documents; manually verify tree correctness for at least one full contract.

Phase 2: Anonymization Layer (3-4 days)

- Integrate spaCy NER; build the entity-to-structural-token mapping with per-session persistence.
- Write the enforcement test: assert no raw entity string reaches any LLM prompt.
- Handle edge cases: nested entities, repeated entities across sections, numeric values (amounts, dates).

Phase 3: Embeddings, Clause Categorization & Drift Engine (5-7 days)

- Integrate sentence-transformers; batch-encode clauses; cache vectors.
- Build the fixed clause-category taxonomy and a first-pass classifier (rule-based or LLM-assisted).
- Implement cross-version clause matching (greedy nearest-neighbor is a reasonable v1) and cosine-similarity drift scoring.
- Calibrate the Cosmetic / Intent-Shift / Substantial-Rewrite thresholds against a small hand-labeled set, rather than hard-coding the blueprint's 0.98/0.85 numbers as ground truth.

Phase 4: Local LLM Integration & Role-Weighted Scoring (5-7 days)

- Build the Ollama client and the strict-JSON prompt templates for classification / plain-language translation / risk explanation.
- Implement the role x category weight matrix (config-driven) and the score-combination logic.
- Add JSON-schema validation and the single-retry-then-fallback behavior for malformed LLM output.

Phase 5: Anchor Verification (Hallucination Guard) (3-4 days)

- Implement the deterministic string/regex match of `quoted_text` against the source clause tree.
- Build a small adversarial test set of deliberately fabricated claims and confirm 100% rejection.
- Wire `verify/` as a hard gate between `llm/` and the UI — nothing bypasses it.

Phase 6: Streamlit UI (4-5 days)

- Upload flow, role selector, single-document annotated view.
- Two-version comparison view with inline drift highlights.
- Session-state caching keyed by file content hash to avoid redundant pipeline runs.
- Basic report export (JSON, optionally PDF).

Phase 7: Evaluation Against CUAD (4-6 days)

- Write the CUAD loader and map its label categories to your clause taxonomy.
- Compute Precision/Recall/F1 per category and an aggregate macro-F1; commit results to the repo.
- Iterate on the classifier/scoring logic based on error analysis, not just the aggregate number.

Phase 8: Polish, Docs & Portfolio Packaging (3-4 days)

- README with architecture diagram, setup steps, and a recorded demo (GIF or short video).
- Clean up config, add docstrings, finalize the resume bullet with real, reproducible metrics from Phase 7.
- Optional stretch: export report, small adversarial-robustness write-up, legal-domain fine-tuned embedding model.

Total estimated duration: roughly 6-8 weeks at a steady self-study pace. Phases 1-2 and 3-4 can each be loosely parallelized in planning (e.g. start reading about embeddings while finishing the parser), but build them in this order — each phase's output is the next phase's input.

4. Things to Learn (Mapped to Phases)

For Phase 1 (Parsing)

- PDF internals: text layers vs. images, font metadata, bounding boxes (pdfplumber / PyMuPDF docs).
- Basic tree/AST data structures in Python (you likely already have this from DSA prep).
- Heuristic design: how to turn "font size + numbering pattern" into reliable rules, and how to test heuristics empirically.

For Phase 2 (Anonymization)

- Named Entity Recognition fundamentals: what spaCy's pipeline actually does, entity types (PERSON, ORG, MONEY, GPE, DATE).
- Precision/recall trade-offs specific to NER (a missed entity is a privacy leak; this differs from generic classification).
- Basic idea of fine-tuning a token-classification model, in case the off-the-shelf spaCy model under-performs on legal text.

For Phase 3 (Embeddings & Drift)

- Sentence embeddings: how sentence-transformers models are trained (contrastive/siamese objectives) and why cosine similarity is the natural metric for them.
- Vector similarity search basics (even at small scale, understanding nearest-neighbor matching matters).
- How to empirically calibrate a threshold: precision-recall curves, not guessing a cutoff number.

For Phase 4 (Local LLM Integration)

- Running local LLMs via Ollama: quantization basics (4-bit vs 8-bit trade-offs in quality/speed/memory).
- Constrained/structured generation: getting reliable JSON out of an LLM, schema validation, retry strategies.
- Prompt engineering specifically for narrow, bounded tasks (classification/extraction) rather than open-ended generation — this is a different skill than chatbot prompting.

For Phase 5 (Anchor Verification)

- Regex and exact/normalized string matching in Python (whitespace/case normalization without introducing fuzzy-match risk).
- Why this pattern is called "grounding" or "attribution verification" in the broader RAG/LLM-reliability literature — useful vocabulary for interviews.

For Phase 6 (UI)

- Streamlit session state and caching (st.cache_data, st.session_state) to avoid redundant recomputation.

- Basic UI/UX principles for side-by-side comparison views (diff-style rendering).

For Phase 7 (Evaluation)

- The CUAD dataset structure and its clause-category label set.
- Precision, Recall, F1 and macro- vs micro-averaging — make sure you can explain the difference confidently in an interview.
- Basic error analysis: reading through false positives/negatives to find systematic weaknesses, not just reporting a single number.

5. Test Plan

5.1 Unit Tests

Module	Test Cases
parser/	Correct nesting for a known sample contract; fallback triggers correctly on a headingless document; page/char-span of
anonymize/	All PERSON/ORG/MONEY/DATE entities in a fixture doc are masked; same entity maps to the same token consistent
embed/	Identical text produces identical vectors (determinism); similar clauses score high similarity, unrelated clauses score low
drift/	A known cosmetic rewrite (paraphrase, same meaning) classifies as Cosmetic; a known 'shall'->'may' style change cla
score/	Same clause + different roles produces different, explainable scores; scoring is deterministic given the same weight m
llm/	Malformed JSON triggers exactly one retry then a graceful 'unclassified' fallback; schema validation rejects a response
verify/	A true, exactly-quoted claim passes; a fabricated claim (text not present in source) is rejected; a claim with only whitesp

5.2 Integration Tests

- Full pipeline run on a real sample contract, end-to-end, asserting the final output structure is well-formed.
- Two-version comparison end-to-end: upload v1, upload v2, confirm the drift classification list is non-empty and internally consistent.
- 'Clear session' action actually removes cached files from the local working directory.

5.3 Adversarial / Reliability Tests

- Hand-craft at least 10 deliberately fabricated risk claims (quotes not present in source, or misattributed clause IDs) and confirm the Anchor Verifier rejects 100% of them.
- Feed a document with unusual formatting (no headings, inconsistent numbering) and confirm the system degrades gracefully (fallback chunking + low-confidence flag) rather than crashing or silently mis-parsing.
- Confirm via network-level inspection (e.g. mock/patch the socket layer in tests) that no outbound call is made containing document text.

5.4 Evaluation / Benchmark Tests

- CUAD-based Precision/Recall/F1 computed and stored as a versioned artifact (e.g. eval_results.json) so improvements over time are trackable.
- Regression test: re-running the eval suite after a code change should not silently drop F1 below a committed threshold (simple CI-style guard, even if you're not running full CI).

6. Suggestions

A few practical notes worth keeping in mind as you build this, based on what tends to matter most for a project like this landing well with recruiters and being genuinely finishable solo:

- **Ship the vertical slice first.** Get one document through the *entire* pipeline (parse → anonymize → score → verify → display) end-to-end early, even with a dumb rule-based classifier instead of the LLM. It's much easier to improve a working pipeline than to assemble eight "finished" modules that have never actually run together.
- **Don't hard-code the blueprint's thresholds as fact.** The 0.98 / 0.85 similarity cutoffs and the role-weight matrix in the blueprint are reasonable starting hypotheses, not measured values — treat calibrating them against real labeled examples (Phase 3, Phase 7) as part of the actual ML work, and say so explicitly in your writeup. That's the difference between "I copied a formula" and "I validated a threshold," which is exactly the kind of distinction a technical interviewer will probe for.
- **The anchor-verification component is your strongest interview talking point.** It's a small amount of code but it directly addresses a real, well-known failure mode of LLM systems (hallucinated citations/claims) with a deterministic, testable guarantee. Be ready to explain it crisply in under a minute, and make sure your adversarial test set (Section 5.3) is real and in the repo, not just described.
- **Keep the CUAD evaluation honest and specific.** Report per-category F1, not just one aggregate number, and be ready to talk about where the system does worse (this is far more convincing in an interview than a single polished score).
- **This project is a strong fit if your goal is a concrete, evaluation-backed, deployable system** rather than another literature survey — it forces real engineering decisions (parsing heuristics, anonymization guarantees, hallucination gating) on top of the ML components, which is exactly the kind of breadth that differentiates a portfolio project from a notebook.
- **Scope discipline matters more than feature count.** The Non-Goals in the PRD (no drafting, no e-signature, no multi-user) are there on purpose — resist adding them even if they seem quick, since they don't strengthen the core technical story and they do extend the timeline.
- **Log everything the Anchor Verifier rejects.** Beyond debugging, a short 'here's what the model tried to hallucinate and how the system caught it' example is a genuinely compelling thing to show live in an interview or demo video.

This document, the PRD, and the TRD are meant to be read together: PRD for what and why, TRD for the system-level how, and this document for the architecture rationale, week-by-week execution plan, what to study when, and how to test it rigorously.